DATA STRUCTURES LAB RECORD

Implementation of Priority Queue using Binary Heap in C

Source Code (PRIORITY_QUEUE.C)

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

typedef struct {
   int items[MAX];
   int size;
} PriorityQueue;

void swap(int* a, int* b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

void heapifyUp(PriorityQueue* pq, int index) {
   if (index && pq->items[(index - 1) / 2] > pq->items[index]) {
       swap(&pq->items[(index - 1) / 2], &pq->items[index]);
       heapifyUp(pq, (index - 1) / 2);
   }
}

void enqueue(PriorityQueue* pq, int value) {
   if (pq->size == MAX) {
       printf("Priority queue is full\n");
       return;
   }
   pq->items[pq->size++] = value;
   heapifyUp(pq, pq->size - 1);
}

int heapifyDown(PriorityQueue* pq, int index) {
   int smallest = index;
   int left = 2 * index + 1;
   int right = 2 * index + 2;

   if (left < pq->size && pq->items[left] < pq->items[smallest])
       smallest = left;

   if (right < pq->size && pq->items[right] < pq->items[smallest])
       smallest = right;

   if (smallest != index) {
       swap(&pq->items[index], &pq->items[smallest]);
       heapifyDown(pq, smallest);
   }
}

int dequeue(PriorityQueue* pq) {
   if (!pq->size) {
       printf("Priority queue is empty\n");
       return -1;
   }
   int item = pq->items[0];
   pq->items[0] = pq->items[--pq->size];
   heapifyDown(pq, 0);
   return item;
}

int peek(PriorityQueue* pq) {
   if (!pq->size) {
       printf("Priority queue is empty\n");
       return -1;
   }
   return pq->items[0];
}

int main() {
   PriorityQueue pq = { { 0 }, 0 };
   enqueue(&pq, 3);
```

Sample Output

```
2 dequeued from queue
Top element is 3
```